

第四届环球杯



第 17 赛段：圣彼得堡大奖赛

February 21-22, 2026

该习题集应包含 13 个问题，共 28 页。

Based on



彼得罗扎沃茨克编程营

编制



圣彼得堡国立大学



问题 A. 蚁丘/蜜罐模拟器

时间限制：2 秒内存限制：1024 MB

有 $N \approx 10^{100}$ 只蚂蚁的蚁丘位于无限平面上。除此之外，该平面仅包含一个半径为 R 的圆形蜜罐，其中心距蚁丘的距离为 d 。

所有蚂蚁最初都位于蚁丘中。然后它们全部开始彼此独立地移动。每只蚂蚁随机选择其移动方向，并以所选方向的速度 $(\Delta t)^{-1/2}$ 移动 Δt 个单位时间，从而行进 $\sqrt{\Delta t}$ 个单位的距离，其中 $\Delta t \approx 10^{-100}$ 是一个很小的数字。然后每只蚂蚁一次又一次地重复整个过程。每当一只蚂蚁位于蜜罐内时，它在蜜罐内停留的每单位时间都会收集一单位的蜂蜜。

您的任务是在开始后的 T 个时间单位内模拟蚁丘和蜜罐，并计算两个数字：在 T 个时间单位后最终进入蜜罐的蚂蚁的分数 f 以及每只蚂蚁收集的蜂蜜的平均数量 h 。

输入

输入的第一行包含一个整数 k ，即测试用例的数量 ($1 \leq k \leq 1000$)。

接下来的每一行 k 描述一个测试用例，并包含三个实数 d 、 R 和 T ，小数点后最多有四位数字 ($0 \leq d \leq 100$; $0.01 \leq R \leq 100$; $0.01 \leq T \leq 100$)。

输出

您必须输出 k 行，每个测试用例一行。每一行应包含 f 和 h 的值，并用一个或多个空格分隔。您必须输出与正确值最多相差 10^{-8} 的 f 值，以及与正确值最多相差 $10^{-8} \cdot T$ 的 h 值。

例子

<i>standard input</i>	<i>standard output</i>
7	0.6321205588285579 0.8515044932240781
0 1 1	0.09157635496981011 0.1733100735950181
2 1 3	0.8767916692090618 0.9555113318223997
1 2 0.999	0.8766185521451777 0.9563880369284812
1 2 1	0.8764454902564369 0.9572645689450815
1 2 1.001	0.3457458387231645 0.4012213611982974
1 1 1	0.9999546000702375 9.999961697595344
0 10 10	

问题 B. 号码缺失

时间限制：2 秒内存限制：1024 MB

给您一个从 1 到 $n+1$ 的 n 整数数组。某些整数可能在数组中出现多次。尽管如此，很明显数组中至少缺少一个从 1 到 $n+1$ 的整数。你的任务是找到任何这样的整数。通常，这很容易，但有一个不幸的问题：您的计算机上没有足够的内存来表示整个数组。

因此，您需要生成一个流算法。非正式地，数组的元素是一一提供给您的，无法随机访问它们。不过，有一个安慰：收到数组的最后一个元素后，您可以重新从头开始读取它。理想情况下，您希望避免这种情况并一次性解决整个问题。然而，有时（包括这个问题）可能需要对输入进行多次传递，以避免使用过多的内存。您需要平衡这两个问题，同时不要使用太多内存，也不要对数组进行太多传递（如果允许您存储 n 位内存或使用 n 传递，那么问题就微不足道了）。

然而，仍然有一个问题。出于技术原因，您需要在字母表 $\{0, 1, \dots, n+1\}$ 上生成确定性有限自动机 (DFA)，以实现算法的逻辑。字母表中的字符对应于您在读取流时收到的信息：从 1 到 $n+1$ 的数字 1 表示从数组中读取相应的整数，而数字 0 表示已到达数组末尾。在前 7 次到达数组末尾后，数组将按照与之前相同的顺序再次提供给您，始终以 0 字符结尾，表示数组的结尾。因此，您的 DFA 将被输入字符串 $a_1, a_2, \dots, a_n, 0$ 总共 8 次（这里， $a_1, a_2, \dots, a_n$ 是隐藏数组）。最后，在执行的任何时刻，您的自动机都可以“说”它已经知道问题的一些正确答案；在这种情况下，执行将停止。如果您的 DFA 在读取整个流（包括末尾的 0 字符）8 次时尚未执行此操作，则不会接受该解决方案。同样，如果它停止了，但答案不正确，它也不会被接受为解决方案。

让我们正式地陈述问题。有一个由 1 到 $n+1$ 之间的 n 整数组成的隐藏数组 $a_1, a_2, \dots, a_n$ 。在示例中为 $n=4$ ，在所有其他测试中为 $n=250$ 。您需要指定一个最多具有 4000 个状态的有限状态机，编号从 0 到 $q-1$ ，其中 $q \leq 4000$ 是状态数。对于状态 i ($0 \leq i \leq q-1$)，您需要指定 $n+2$ 个数字 $\delta(i, 0), \delta(i, 1), \dots, \delta(i, n+1)$ 。数字 $\delta(i, j)$ 表示 DFA 处于状态 i 时的行为，并从输入中读取数字 j 。它必须是从 $-(n+1)$ 到 $q-1$ （含）之间的整数。负整数对应于执行停止并给出答案的事实；具体来说， $-s$ 对应于您的 DFA，声称 $+s$ 是问题的正确答案。非负整数对应于继续执行并且自动机的状态从 i 变为 $\delta(i, j)$ 的事实。自动机的执行从状态 0 开始。

如果您的 DFA 在读取字符串 $a_1, a_2, \dots, a_n, 0, a_1, a_2, \dots, a_n, 0, \dots, a_1, a_2, \dots, a_n, 0$ （其中数组重复 8 次）时停止并给出正确答案，则认为您的 DFA 在数组 $a_1, a_2, \dots, a_n$ 上正常工作。如果您的 DFA 在所有可能的 1 到 $n+1$ 之间的整数数组 $a_1, a_2, \dots, a_n$ 上都能正确工作，则一般认为它可以正常工作（因此，不鼓励使用概率解决方案）。

对于 $n=4$ ，我们将在所有可能的数组上检查您的 DFA。对于 $n=250$ ，我们没有办法在所有阵列上检查它，因此我们将在少于 5000 个专门选择的阵列上检查它。该问题有两个测试；这两个测试都有效地检查您的方法是否同时在多个阵列上正常工作。

输入

一个整数 n ，等于 4 或 250。

输出

o 在第一行中，打印一个整数 q ($1 \leq q \leq 4000$) 表示 DFA 中的状态数。



在以下 q 行的第 i 行中，打印 $n + 2$ 个整数 $\delta(i - 1, 0)$ 、 $\delta(i - 1, 1)$ 、 $\dots$ 、 $\delta(i - 1, n + 1)$ 。它们都应该从 $-(n + 1)$ 到 $q - 1$ （含）并且对应于自动机的转换。请注意，所有数字都应在此范围内：即使在任何有效输入上永远无法达到状态 i 和输入数字 j 的某种组合，您指定的 $\delta(i, j)$ 值仍应在 $[-(n + 1), q - 1]$ 范围内。

例子

<i>standard input</i>	<i>standard output</i>
4	8 -3 0 0 1 0 0 2 1 1 1 1 1 -1 3 2 2 2 2 4 3 3 3 3 3 -5 4 4 4 4 5 6 5 5 5 5 5 -2 6 7 6 6 6 -4 7 7 7 7 7

笔记

该示例的答案是一个非常无聊的自动机，它使用四遍来检查数组是否包含数字 3、1、5 和 2（按该顺序）。每个遍实际上只使用两个状态，这两个状态对应于它是否已经找到了它在当前遍中寻找的数字。如果它到达数组末尾（字符 0）而没有找到所需的数字，则它立即停止，表示所需的数字确实丢失了。否则，它将继续使用下一个通道来搜索下一个数字。最后，如果它找到了所有四个必要的整数，那么数字 4 一定会丢失。因此，它不会进行第五遍寻找数字 4（但它可以在不违反自动机状态和使用的遍数的限制的情况下进行）。

当然，这种幼稚的方法不可能适用于 $n = 250$ ，并且需要更好的方法。该示例存在的主要原因是为了说明输出格式。

问题 C. 弦乐研讨会

时间限制: 5.5 秒内存限制: 1024 MB

克利奥帕特拉喜爱弦乐。当给她一个长度为 n (、由字符 $s[1]$ 组成的字符串 s 时, $\dots, s[n]$), 她立即开始排序。在排序过程中, 她执行一系列 $s[i] \leftrightarrow s[i+1]$ 形式的交换, 其中 $i \in \{1, 2, \dots, n-1\}$ 。最后, 她返回排序后的字符串, 并要求所有交换的金币数量等于 i 的总和 (如果使用相同的 i 进行多次交换, 则 i 将计入与交换次数相同的总和中)。克利奥帕特拉并不浪费: 在所有对字符串进行排序的方法中, 她选择了她收到的用于排序的硬币数量最少的方法。

凯瑟琳·德·美第奇也喜欢弦乐。她有一个长度为 n 的字符串, 她想玩它。具体来说, 她对以下操作感兴趣:

1. “ $1\ i\ c$ ” — 将 $s[i]$ 替换为 c ; 2. “ $2\ i\ j$ ” — 获取从第 i 到第 j 个字符 (包括 $i \leq j$) 的子字符串 s 的副本并对其进行排序, 而字符串 s 的字符保持不变。
3. “ $3\ i\ j$ ” — 从第 i 到第 j 个字符 (包括 $i \leq j$) 中提取子字符串 s , 对其进行排序, 然后将其插回到字符串 s 的同一位置。

凯瑟琳·德·美第奇自己可以处理第一类询问, 但第二类和第三类询问太复杂了, 她会去找克利奥帕特拉。在评估此过程的成本时, Cleopatra 将不会对字符串 s (从 i 到 j) 的索引进行求和, 而是对相对于子字符串开头的索引 (从 1 到 $j-i+1$) 进行求和。

凯瑟琳·德·美第奇也不浪费, 并仔细监控她的预算。因此, 对于她向克利奥帕特拉提出的每一个请求, 计算一下凯瑟琳·德·美第奇需要花费多少金币。

输入

输入的第一行包含一个整数 t - 测试用例的数量 ($1 \leq t \leq 3 \cdot 10^5$)。对于每个测试用例:

第一行包含两个整数, n 和 q - 字符串的长度和查询数量 ($1 \leq n, q \leq 3 \cdot 10^5$)。下一行包含一串 n 大写英文字母。以下 q 行描述了查询。每个查询描述都具有以下三种类型之一:

1. “ $1\ i\ c$ ” — 将 $s[i]$ 替换为 c ($1 \leq i \leq n$; $c \in \{A, B, \dots, Z\}$);
2. “ $2\ i\ j$ ” — 求在字符串 s 保持不变的情况下对子字符串 $s[i..j]$ 进行排序的成本 ($1 \leq i, j \leq n$);
3. “ $3\ i\ j$ ” — 求对子字符串 $s[i..j]$ 进行排序的成本, 同时将字符串 s 中的子字符串 $s[i..j]$ 替换为排序后的子字符串 ($1 \leq i, j \leq n$)。

在每个测试用例中, 至少会有一个第二或第三类型的查询。

所有测试用例的 n 总和最多为 $3 \cdot 10^5$ 。所有测试用例的 q 总和最多为 $3 \cdot 10^5$ 。

输出

对于每个测试用例, 在单独的行中输出第二和第三类型的所有查询的答案, 在是, 克利奥帕特拉将收到的每种类型的硬币数量。



<i>standard input</i>	<i>standard output</i>
1	18
5 20	3
DBCAA	1
2 1 5	16
2 2 4	0
1 3 Z	0
2 1 3	7
3 1 5	1
2 1 5	1
1 5 A	1
3 2 4	9
2 1 5	0
1 1 C	0
2 1 2	3
3 4 5	3
2 3 5	
1 2 B	
2 1 5	
3 1 1	
2 1 1	
1 4 D	
2 1 5	
3 1 5	
3	4
3 5	4
CBA	0
2 1 3	0
3 1 3	0
2 1 3	9
1 2 C	9
2 1 2	0
5 5	0
ABCDE	3
2 2 5	1
1 5 A	1
2 1 5	
3 1 5	
2 1 5	
6 5	
YYYYYY	
2 1 6	
1 3 A	
2 1 6	
3 2 5	
2 1 6	

笔记

c 考虑第一个例子。其中，初始字符串是“DBCAA”。让我们按顺序浏览所有 20 个查询：

1. 查询“2 1 5”。当前字符串：“DBCAA”。子串 $t[1..5]$ = “DBCAA”，答案：18。字符串不变。
2. 查询“2 2 4”。字符串：“DBCAA”。子串 $t[2..4]$ = “BCA”，答案：3。字符串不变。
3. 查询“1 3 Z”。更新： $t[3] \leftarrow “Z”$ 。原来是“DBCAA”，现在是“DBZAA”。
4. 查询“2 1 3”。字符串：“DBZAA”。子串 $t[1..3]$ = “DBZ”，答案：1。字符串不变。
5. 查询“3 1 5”。字符串：“DBZAA”。子串 $t[1..5]$ = “DBZAA”，答案：16。变异后，对子串进行排序：“DBZAA” $\rightarrow$ “AABDZ”。该字符串变成“AABDZ”。
6. 查询“2 1 5”。字符串：“AABDZ”。子串“AABDZ”，答案：0。字符串不变。
7. 查询“1 5 A”。原来是“AABDZ”，现在是“AABDA”。
8. 查询“3 2 4”。字符串：“AABDA”。子串 $t[2..4]$ = “ABD”，答案：0。排序后“ABD”不改变，字符串仍然是“AABDA”。
9. 查询“2 1 5”。字符串：“AABDA”。子串“AABDA”，答案：7。字符串不变。
10. 查询“1 1 C”。原来是“AABDA”，现在是“CABDA”。
11. 查询“2 1 2”。字符串：“CABDA”。子串 $t[1..2]$ = “CA”，答案：1。字符串不变。
12. 查询“3 4 5”。字符串：“CABDA”。子串 $t[4..5]$ = “DA”，答案：1。变异后：“DA” $\rightarrow$ “AD”。该字符串变成“CABAD”。
13. 查询“2 3 5”。字符串：“CABAD”。子串 $t[3..5]$ = “BAD”，答案：1。字符串没有改变。
14. 查询“1 2 B”。原来是“CABAD”，现在是“CBBAD”。
15. 查询“2 1 5”。字符串：“CBBAD”。子字符串“CBBAD”，答案：9。字符串不变。
16. 查询“3 1 1”。字符串：“CBBAD”。子字符串 $t[1..1]$ = “C”，答案：0。对长度 1 进行排序不会改变任何内容，字符串仍然是“CBBAD”。
17. 查询“2 1 1”。字符串：“CBBAD”。子串 $t[1..1]$ = “C”，答案：0。字符串不变。
18. 查询“1 4 D”。原来是“CBBAD”，现在是“CBBBD”。
19. 查询“2 1 5”。字符串：“CBBBD”。子串 $t[1..5]$ = “CBBBD”，答案：3。字符串不变。
20. 查询“3 1 5”。字符串：“CBBBD”。子串 $t[1..5]$ = “CBBBD”，答案：3。突变后：“CBBBD” $\rightarrow$ “BBCDD”。最终字符串是“BBCDD”。



This page is intentionally left blank

问题 D. 可区分分布

时间限制：1 秒内存限制：1024 MB

这是一个交互问题。

懒惰的 Sam 和勤奋的 Sarah 的任务是针对字母表 $\{H, T\}$ 上长度为 n 的字符串集提出两个有趣的分布。懒惰的 Sam 将简单地掷硬币 n 次（每一面出现的概率为 $\frac{1}{2}$ ；所有硬币翻转都是独立的），并为每个正面写下“H”，为每个背面写下“T”。你想帮助勤奋的 Sarah 给懒惰的 Sam 一个教训。给你一个整数 t 。帮助 Sarah 提出一个接近于均匀分布的 t 分布，同时以交互方式证明您可以通过长度为 t 的 Oracle 访问来区分懒惰的 Sam 和勤奋的 Sarah 的分布。

形式上，对于某个整数 n ，考虑大小为 2^n 的集合 $S = \{H, T\}^n$ ，该集合由两个字符的字母表上长度为 n 的所有可能字符串组成：“H”和“T”。集合 S 上的分布定义为长度为 2^n 的非负实数 $\{a_s\}_{s \in S}$ 数组，由 S 的元素索引，使得它们的总和等于 1。最简单的分布之一是均匀分布，其中所有 a_s 都等于 2^{-n} 。

对于一对数字 $\varepsilon \in [0; +\infty)$ 和 $k \in \{0, 1, \dots, n\}$ ，我们说 $\{a_s\}_{s \in S}$ 是与 (ε, k) 无关的分布，如果对于 k 索引 $1 \leq i_1 < i_2 < \dots < i_k \leq n$ 的任何选择以及字母表 $\{H, T\}$ 上长度为 k 的字符串的任何子集 $T \subseteq \{H, T\}^k$ ，以下公式成立： $2^{n-k}|T|$ 字符串上的 a_s 之和子序列 $s[i_1]s[i_2] \dots s[i_k]$ 属于 T 的 s 与 $\frac{|T|}{2^k}$ 相差不超过 ε 。用概率术语来说，这可以表示为：

$$\forall 1 \leq i_1 < i_2 < \dots < i_k \leq n \quad \forall T \subseteq \{H, T\}^k \quad \left| \frac{|T|}{2^k} - \mathbb{P}_{s \leftarrow \{a_s\}_{s \in S}} \{s[i_1]s[i_2] \dots s[i_k] \in T\} \right| \leq \varepsilon.$$

直观上，这意味着要区分分布和均匀分布，您要么需要区分差异不超过 ε 的概率，要么查看分布生成的字符串的超过 k 位。

如果分布 $\{a_s\}_{s \in S}$ 与 $(2^{-t}, t)$ 无关，则称为 t -接近均匀分布。

在 (ε, k) 独立性的定义中，暗示您必须首先选择 k 索引，然后才研究这些 k 索引的概率分布。在长度为 k 的 oracle 访问模型中，一切都不同了。生成长度为 n 的随机字符串，然后进行以下格式的 k 轮：您调用一个整数 $i \in \{1, 2, \dots, n\}$ ，作为响应，您将被告知 $s[i]$ 。您可以考虑之前提出的所有问题及其答案，决定接下来调用哪个索引。

为了证明您可以区分懒惰的 Sam 和勤奋的 Sarah 的分布，您需要对长度为 t 的 oracle 访问从其分布中采样的长度为 n 的 m 字符串（每个字符串均独立且统一地从懒惰的 Sam 或勤奋的 Sarah 的分布生成），对它们中的每个人进行结论，并正确猜测至少 $0.75m - 2\sqrt{m}$ 次。

交互协议

第一行包含一个整数 t ，表示所需的均匀分布接近度和预言机访问的长度（ $1 \leq t \leq 5$ ）。

作为响应，打印一行，其中包含单个整数 n ，表示您准备为其提供分布和可区分性的交互式证明的字符串的长度（ $t \leq n \leq 20$ ）。

在下一行中，打印一行包含 2^n 整数 b_s 的行，定义您的分布（ $0 \leq b_s < 2^{64}$ ； $\sum_{s \in S} b_s > 0$ ）。根据这些数据，评审团将根据规则 $a_s = b_s / \sum_{s \in S} b_s$ 选择分布 $\{a_s\}_{s \in S}$ 。 b_s 的值必须按字符串 s 的字典顺序列出：例如，对于 $n = 3$ ，数字应按以下顺序打印： b_{HHH} 、 b_{HHT} 、 b_{HTH} 、 b_{HTT} 、 b_{THH} 、 b_{THT} 、 b_{TTH} 、 b_{TTT} 。

接下来，读取一个整数 m ，表示交互式检查的数量（ $1 \leq m \leq 10^4$ ）。在此之后， m

检查将是 提出，需要一个接一个地完成 其他。

在每次检查开始时，陪审团将首先以 $\frac{1}{2}$ 的概率从懒惰的 Sam 或勤奋的 Sarah 中选择分布 $\{a_s\}_{s \in S}$ 。之后，陪审团采样一个随机字符串 $s \sim \{a_s\}_{s \in S}$ ：任何字符串 s 都可以以概率 a_s 获得。然后，陪审团让您有机会就字符串 s 的字符提出不超过 t 个问题。要提出问题，请打印一行包含整数 $q \in \{1, 2, \dots, n\}$ 的行，表示感兴趣的字符的索引。之后，读出字符“H”或“T”，这就是查询的答案。问完从零到 t 的问题后，如果您认为 s 取自懒惰的 Sam 的发行版，则打印一行名为“Sam”的行；如果您认为 s 取自勤奋的 Sarah 的发行版，则打印一行名为“Sarah”的行。此后，立即进行下一项检查，直到完成所有 m 。

在任何情況下都可以打印字符串。

后 每打印一行，不要忘记刷新输出 缓冲。

例子

<i>standard input</i>	<i>standard output</i>
2	3 7 4 8 9 7 6 7 1
3	1
T	2
T	SAM
	1
T	2
T	Sarah
	3
T	sARaH



问题 E. 橡树周围的四位圣人

时间限制: 1.5 秒
内存限制: 1024 MB

这是一个运行两次的问题；两次运行都是交互式的。

一棵宽大的橡树周围站着四位贤者。每个圣人头上都戴着一顶红色、绿色或蓝色的帽子。每个圣人可以看到邻居帽子的颜色，但看不到自己的帽子颜色（帽子没有挂在他们的眼睛上）或对面圣人的帽子颜色（被宽阔的橡树挡住了）。每个圣人必须在不与其他人交流的情况下，在一张纸上写下他们帽子的颜色。制定一个协议，使得四位圣人中至少有一位一定能猜出他们帽子的颜色。

交互协议

第一行包含两个整数 t 和 s ，表示测试用例的数量和解决方案的运行次数（ $1 \leq t \leq 100$; $s \in \{1, 2\}$ ）。接下来是 t 测试用例的描述，每个测试用例占一行。

如果 $s = 1$ ，则测试用例由四个单词组成： c_1 、 c_2 、 c_3 和 c_4 ，表示参与者帽子的颜色（ $c_i \in \{\text{red, green, blue}\}$ ）。作为响应，输出一行，其中包含单个整数 m （ $1 \leq m \leq 4$ ），表示根据您的协议猜测其帽子颜色的圣人的编号（如果有多个，则输出任何一个）。

如果 $s = 2$ ，则测试用例由一个整数 m 和两个单词 ℓ 和 r 组成，表示必须猜测其帽子颜色的圣人的编号，以及其左右邻居帽子的颜色（ $1 \leq m \leq 4$; $\ell, r \in \{\text{red, green, blue}\}$; $\ell = c_{(m+2) \bmod 4+1}$; $r = c_{m \bmod 4+1}$ ）。作为响应，输出一行包含单词 g 的行，该单词是第 m 个圣人的帽子颜色。如果 $g = c_m$ ，答案将被接受。交互器不区分大小写，因此您可以以任何大小写（大写或小写）打印每个字母。

防止信息传递

在测试用例之间，陪审团采取了以下措施

g措施:

- 两次运行都是交互式的。因此，在输出所有先前测试用例的答案之前，您将不会收到下一个测试用例。打印每一行后，不要忘记刷新输出流（例如，在 C++ 中使用 `fflush(stdout)` 或 `cout.flush()`、Python 中的 `sys.stdout.flush()` 或 Java 或 Kotlin 中的 `System.out.flush()`）。否则，您可能会收到超出空闲限制的错误。
- 在第二次运行期间，陪审团可以按任意顺序重新排列 t 测试用例，这可能与它们在第一次运行中出现的顺序不匹配。
- 在第二次运行期间，陪审团还可能添加与第一次运行中的任何测试用例都不对应的伪造测试用例。对于这些情况，任何“红色”、“绿色”或“蓝色”的答案都将被视为正确；需要它们的目的是为了让您的程序无法轻松区分真实的测试用例和伪造的测试用例并在用例之间传递信息。因此，第二次运行中的 t 可能超过第一次运行中的 t 。真假案件总数不能超过 100 件。



例子

<i>standard input</i>	<i>standard output</i>
1 1 green red blue red	2
4 2 2 blue green 4 red blue 3 red red 4 red blue	blue BLUE bLUE Blue



问题 F. 优点和缺点

时间限制：3 秒内存限制：1024 MB

米沙在笔记本上写下几个他认为漂亮的整数，然后去玩弄正负。他认为，如果一串正负号的两个符号数量相等并且以大量负数结尾，则该串正负数是平衡的。长度为 $2, 4, \dots, 2n$ 的平衡串有多少个？答案可能非常大，因此请对 998 244 353 求模。

输入

第一行包含两个整数， n 和 m ：字符串的最大长度和 Misha 笔记本中整数的数量 ($1 \leq n, m \leq 200\,000$)。

下一行包含 $[0, n]$ 范围内的 m 个不同的非负整数：Misha 笔记本中的数字。

输出

输出 n 行，答案以 998 244 353 为模。

例子

<i>standard input</i>	<i>standard output</i>
5 3 1 2 3	1 3 10 34 120



This page is intentionally left blank



问题 G. 交通灯

时间限制: 5.5 秒
内存限制: 1024 MB

给你一棵树，每个顶点都有一个发出红色、黄色或绿色光芒的灯泡。回答人以下类型的查询：

y

- “1 v c ” ——将顶点 v 处的灯泡颜色更改为颜色 c ;
- “2 v ” ——给定顶点 v ，保证发出黄色光，找到树中经过 v 的最短红绿灯，并输出其中的边数。红绿灯被定义为树中的一条简单路径，其一端为红色，另一端为绿色，并且该路径的至少一个内部顶点为黄色。

输入

输入的第一行包含一个整数 t ，即测试用例的数量 ($1 \leq t \leq 2 \cdot 10^5$)。对于每个测试用例：

第一行包含两个整数， n 和 q - 树中的顶点数和查询数
($1 \leq n, q \leq 2 \cdot 10^5$)。

英语

接下来的 $n - 1$ 行中的每一行都包含一对整数， u_i 和 v_i - 树的边的末端 ($1 \leq u_i, v_i \leq n$; $u_i \neq v_i$)。保证这些边形成一棵树。

以下行包含长度为 n 的字符串，由字母 “R”、“Y”、“G” 组成，其中第 i 个字母表示第 i 个顶点处灯泡的初始颜色。“R” 代表红光，“Y” 代表黄光，“G” 代表绿光

嗯

n。

然后按照 q 查询的描述进行操作，每行一个：

- “1 v c ” ——将顶点 v 处的灯泡颜色更改为颜色 c ($1 \leq v \leq n$; $c \in \{R, Y, G\}$);
- “2 v ” ——输出经过 v ($1 \leq v \leq n$) 的最短红绿灯的长度。保证在执行此查询时，顶点 v 处的灯泡发出黄色光。保证至少有一个该类型的查询。

保证所有测试用例的 n 之和不超过 $2 \cdot 10^5$ 。保证所有测试用例的 q 之和不超过 $2 \cdot 10^5$ 。

输出

对于每个测试用例和 “2 v ” 类型的每个查询，输出一个数字 d - 树中最短简单路径上的边数，该路径从红色顶点开始，穿过黄色顶点 v ，并以绿色顶点结束。如果这样的路径不存在，则输出 -1 。



例子

<i>standard input</i>	<i>standard output</i>
1 7 7 1 2 2 3 2 4 4 5 4 6 6 7 RYGYRGY 2 2 1 7 G 2 4 1 1 Y 2 2 1 5 Y 2 4	2 2 3 -1
3 3 2 1 2 2 3 YRG 2 1 1 1 Y 4 3 1 2 2 3 2 4 RYGY 2 2 1 1 Y 2 4 5 8 1 2 2 3 3 4 3 5 YRYGY 2 3 1 5 G 2 1 1 2 Y 2 3 1 1 R 2 3 1 4 Y	-1 2 -1 2 -1 -1 3

问题 H. 剩余的甜蜜!

时间限制: 2 秒内存限制: 1024 MB

给出整数 $n, m > 0$ 以及整数 $a_1, a_2, \dots, a_n \in \{-1, 0, 1, \dots, m-1\}$ 。构造一棵具有 n 个顶点 $V = \{v_1, v_2, \dots, v_n\}$ 的树 T , 条件如下: 对于每个顶点 v_i (其中 $a_i \neq -1$), 度数 $\deg_T(v_i)$ 除以 m 后的余数必须等于 a_i 。如果 $a_i = -1$, 则对 v_i 的次数没有限制。

输入

输入的第一行包含一个整数 t , 即测试用例的数量 ($1 \leq t \leq 5 \cdot 10^5$)。对于每个测试用例 e
: 第一行包含两个整数, n 和 m , 表示树中的顶点数和模数 ($1 \leq n \leq 5 \cdot 10^5; 1 \leq m \leq 2 \cdot 10^9$)。

下一行包含 n 整数 $a_1, a_2, \dots, a_n$ 表示所需的余数 ($-1 \leq a_i < m$)。

所有测试用例的 n 之和不超过 $5 \cdot 10^5$ 。

输出

对于每个测试用例, 如果存在这样一个具有 n 顶点的树, 则输出 “YES” (在任何情况下), 然后是 $n-1$ 行, 其中有两个整数, 每个整数表示每条边的端点。否则, 输出 “NO” (无论如何)。

例子

<i>standard input</i>	<i>standard output</i>
6	YES
1 5	YES
0	1 2
2 3	YES
1 1	1 2
5 4	1 3
0 1 1 1 1	1 4
6 2	1 5
1 0 0 0 0 1	YES
3 2	1 2
1 1 1	2 3
4 10	3 4
3 3 1 1	4 5
	5 6
	NO
	NO

笔记

在第一个测试用例中, 我们被要求构建一棵具有 $n = 1$ 个顶点、 $m = 5$ 和度残差数组 $b = [0]$ 的树。只有一个顶点, 不可能有边, 因此恰好存在一棵可能的树: 度为 0 的单个顶点。由于 $0 \bmod 5 = 0$ 满足要求; 因此答案是 “是”。

在第二个测试用例中, 我们需要一棵位于 $n = 2$ 顶点、 $m = 3$ 和残数 $b = [1, 1]$ 的树, 这意味着两个度数必须全等于 1 模 3。在两个顶点上, 只有一棵树: 单边 1–2。它的度数是 (1, 1) 并且实际上是 $1 \bmod 3 = 1$, 所以答案是 “YES”, 并且打印 “1 2” 是有效的。

在第三个测试用例中, 我们需要 $n = 5$ 、 $m = 4$, 以及残数 $b = [0, 1, 1, 1, 1]$: 第一个顶点的度数必须可被 4 整除, 而其他四个顶点的度数必须为 $\equiv 1 \pmod{4}$ 。自然构造是以顶点 1 为中心的星形: 将 1 连接到所有其他顶点。然后 $\deg(1) = 4$ 和 $4 \bmod 4 = 0$, 而



2, 3, 4, 5 中的每一个都有度 1 和 $1 \bmod 4 = 1$ 。因此，答案是“是”，边“1 2”、“1 3”、“1 4”、“1 5”起作用。

在第四个测试用例中，我们需要一棵具有 $n = 6$ ， $m = 2$ ，残基 $b = [1, 0, 0, 0, 0, 1]$ 的树，即端点必须具有奇数度，四个中间顶点必须具有偶数度。模 2，这纯粹是奇偶校验约束。简单路径 1-2-3-4-5-6 完美拟合：顶点 1 和 6 的度数为 1（奇数），顶点 2, 3, 4, 5 的度数为 2（偶数）。因此答案是“是”，并且路径边缘是正确的。

在第五个测试用例中，我们被要求在 $n = 3$ 上用 $m = 2$ 和 $b = [1, 1, 1]$ 构建一个图（特别是一棵树），因此每个顶点必须具有奇数度。这是不可能的：在每个无向图中，奇数度顶点的数量始终是偶数，因此任何一组边都不会出现三个奇数度。因此，判决为“否”。

在第六个测试用例中，我们有 $n = 4$ 、 $m = 10$ 、 $b = [3, 3, 1, 1]$ 。由于 $m > n - 1 = 3$ ，模 10 的余数等于度本身，因此我们需要精确的 $\deg = [3, 3, 1, 1]$ 。这迫使两个顶点与所有其他顶点相邻（4 顶点图中的度为 3），并且当 $n > 2$ 时，任何这样的对都必须与任何第三个顶点一起形成一个循环。树不能包含循环，因此结论也是“否”。



问题一、木方格

时间限制：2 秒内存限制：1024 MB

这是一个交互问题。

Vlad 有一个空的有向图，其中 n 个顶点由 1 到 n 的整数标记。他计划将 $n - 1$ 条边一条一条地添加到该图中，确保在每一时刻都满足两个条件：

1. 该图是有向森林：每个连通分量都是有向根树的形式，其中边从根指向叶子。
2. 对于图的每个顶点，从该顶点可达的顶点集合是一段无间隙的整数。

你的 task 是在添加每个条件后检查这两个条件边缘。

交互协议

首先，陪审团程序将提供一个带有整数 n ($1 \leq n \leq 2 \cdot 10^5$) 的字符串，即顶点数。
然后，将有 $n - 1$ 条边添加。对于每次添加，评审团将输入一个带有顶点编号 v 和 u ($1 \leq v, u \leq n$) 的字符串； $v \neq u$ 。添加边 $v \rightarrow u$ 后检查 Vlad 的条件是否满足，并按以下格式在单独的行中输出答案。

- 如果不满足第一个条件，则输出字符串 “Bad orienting Forest”。
- 如果不满足第二个条件，则输出字符串 “Bad segment at v ”，其中 v 是不满足条件的任何顶点的编号。
- 如果两个条件都不满足，则输出上述两条消息中的任意一条。
- 否则，如果两个条件都满足，则输出字符串 “Good”。

如果不满足任何条件，您的程序应该在输出相关消息后立即终止，即使它处理的边数少于 $n - 1$ 个。

没有有向边 $v \rightarrow u$ 添加两次。所有 $n - 1$ 边的添加都是预先固定的。

例子

<i>standard input</i>	<i>standard output</i>	Notes
6 1 2 3 4 3 1 5 6 5 1	Good Good Good Good Bad segment at 5	



This page is intentionally left blank

Problem J. Euler Line

Time limit: 1 second
Memory limit: 1024 mebibytes

Given a line on the plane, construct a triangle with vertices at integer points such that its Euler line coincides with the given line.

Recall the basic definitions. Let $\triangle ABC$ be a non-degenerate triangle on the plane, M_A , M_B , and M_C be the midpoints of sides BC , AC , and AB , respectively, and s_A , s_B , and s_C be the *perpendicular bisectors* to sides BC , AC , and AB . Formally, s_A is the perpendicular to line BC , drawn at point M_A ; lines s_B and s_C are defined similarly. From school geometry, it is known that lines s_A , s_B , and s_C intersect at a single point O . The point O has a remarkable property, namely, it is equidistant from all three vertices A , B , and C . The circle centered at O with radius OA is called the *circumcircle* of triangle ABC , and it is the unique circle in the plane that passes through all three vertices of the triangle. The point O is referred to as the *circumcenter* of triangle ABC .

Let H_A , H_B , and H_C denote the orthogonal projections of vertices A , B , and C onto lines BC , AC , and AB , respectively. The segments AH_A , BH_B , and CH_C are called the *heights* of triangle ABC . From school geometry, it is known that lines AH_A , BH_B , and CH_C intersect at a single point H , called the *orthocenter* of triangle ABC . In an acute or right triangle, H lies on the heights themselves, while in an obtuse triangle, it lies on their extensions.

The segments AM_A , BM_B , and CM_C are called the *medians* of the triangle. From school geometry, it is known that the medians intersect at a single point G , called the *centroid* or *barycenter* of triangle ABC .

The triangle $M_A M_B M_C$ is called the *medial* triangle for triangle ABC . The concepts above also apply to it: thus, point G is the barycenter of the medial triangle, and O is the orthocenter of the medial triangle. The circumcircle of triangle $M_A M_B M_C$ passes through M_A , M_B , M_C , H_A , H_B , H_C , as well as through the midpoints of segments AH , BH , CH , which is why it is called the *nine-point circle* of triangle ABC , and its center is denoted as O_9 .

In an equilateral triangle, points O , H , G , and O_9 coincide. In any other triangle, they are pairwise distinct, but they lie on a single line, called the *Euler line* of triangle ABC . The arrangement of these four points on the Euler line is also known: O_9 is always the midpoint of segment OH , and point G lies on segment OH and divides it in the ratio 2 : 1 (with G being twice as close to O as to H).

You are required to construct a non-degenerate triangle ABC with vertices at integer points, such that O , H , M , and O_9 lie on the given line, or report that this is impossible.

Input

The first line contains an integer t , the number of test cases ($1 \leq t \leq 5 \cdot 10^5$).

In the only line of the test case description, there are three integers, k , ℓ , and m , denoting the coefficients of the line ($-10^3 \leq k, \ell, m \leq 10^3$). The line will be defined as the set of points (x, y) that satisfy the equation $kx + \ell y + m = 0$. It is guaranteed that $k^2 + \ell^2 > 0$.

Output

For each test case, output six integers A_x , A_y , B_x , B_y , C_x , C_y , denoting the coordinates of the vertices of a non-degenerate triangle. The conditions $-10^6 \leq A_x, A_y, B_x, B_y, C_x, C_y \leq 10^6$ must be satisfied. The Euler line of triangle ABC must have the equation $kx + \ell y + m = 0$. If such a triangle does not exist, output six zeros separated by spaces.

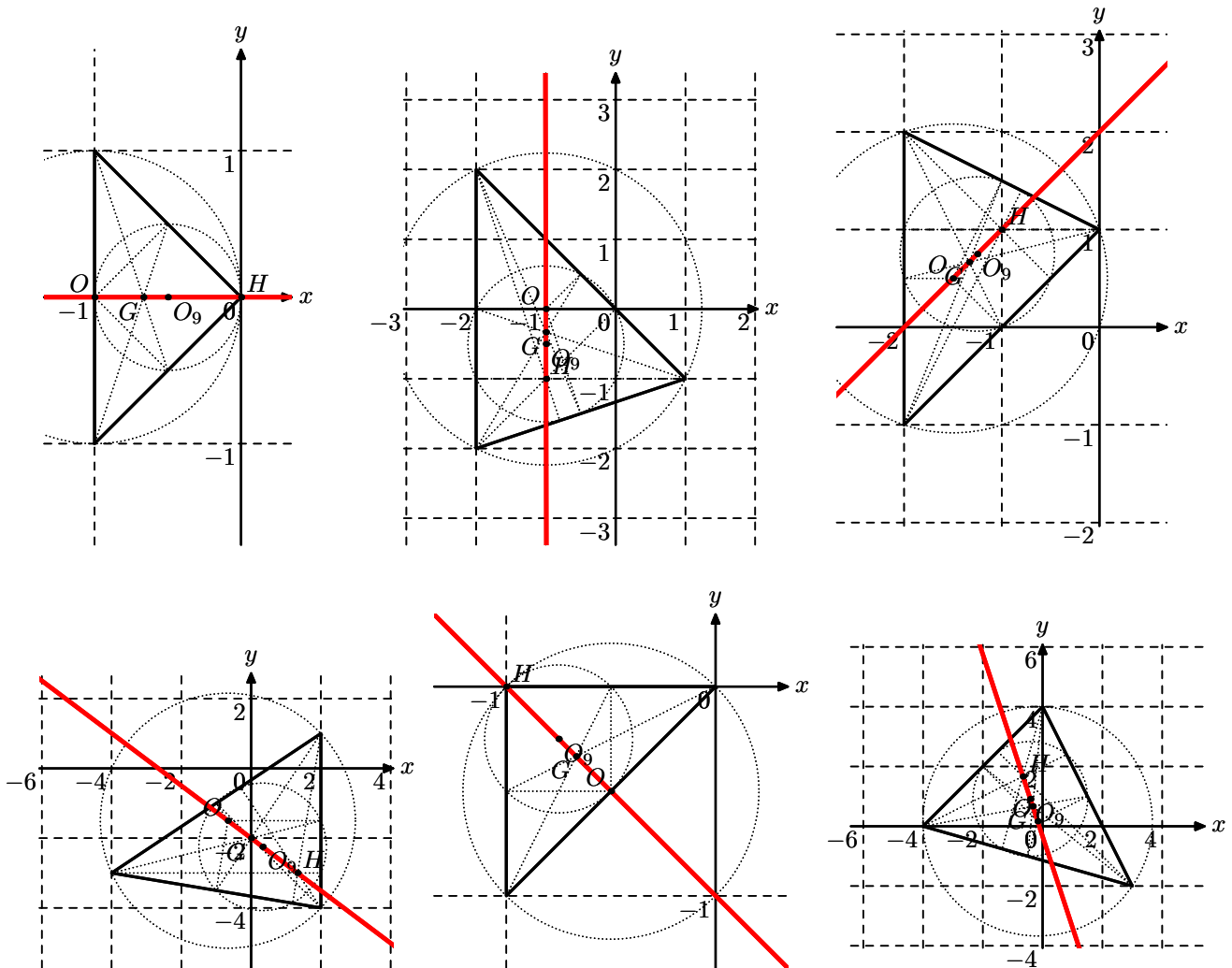
In each of the tests provided by the jury, it is guaranteed that, if there exists at least one triangle ABC with integral vertices with the given Euler line, there is also a triangle with conditions $-10^6 \leq A_x, A_y, B_x, B_y, C_x, C_y \leq 10^6$ satisfied.

Example

<i>standard input</i>	<i>standard output</i>
8	-1 -1 -1 1 0 0
0 1 0	-2 -2 -2 2 1 -1
1 0 1	-2 -1 -2 2 0 1
-1 1 -2	-4 -3 2 -4 2 1
3 4 8	-1 -1 -1 0 0 0
1 1 1	0 0 0 0 0 0
8 4 2	-4 0 3 -2 0 4
27 9 3	5811 -1154 -3261 -2058 -3713 2478
-544 862 12	

Note

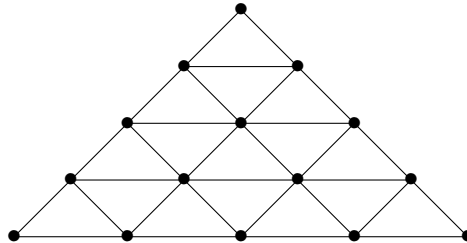
We do not explicitly require $\triangle ABC$ to be non-equilateral because it is impossible to come up with an equilateral triangle whose vertices have integer coordinates.



Problem K. Traversal of a Triangular Grid

Time limit: 2 seconds
Memory limit: 1024 mebibytes

Let $n \geq 1$. Consider the following standard way to divide a large triangle into n^2 smaller triangles by drawing three sets of pairwise parallel lines:



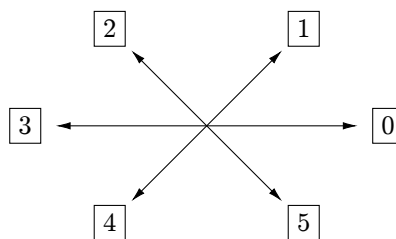
Naturally, this defines an undirected graph with $(n+1)(n+2)/2$ vertices and $3n(n+1)/2$ edges: the vertices are the nodes of the grid or, in other words, the intersection points of the lines (marked in bold in the picture), and the edges are the segments between neighboring nodes. Your task is to construct an Eulerian cycle on the edges of this graph that “starts” from the topmost vertex of the triangle. In other words, you need to start at the topmost vertex, move along the edges, traverse each edge exactly once, and, in the end, return to the topmost vertex. It can be proven that a solution always exists.

Input

An integer n is given, ranging from 1 to 20.

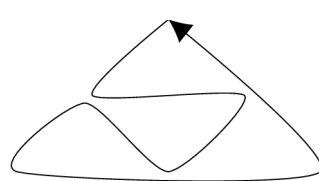
Output

Output a string of exactly $3n(n+1)/2$ digits from 0 to 5, representing some Eulerian cycle of the graph in the following format: 0 means moving directly to the right from the current vertex, 1 means moving in the right-up direction, 2 means moving left-up, 3 means moving left, 4 means moving left-down, and 5 means moving right-down. For clarity, the directions are shown in the picture below.



The traversal **must start from the top vertex**. Any of the possible traversals will be accepted.

Example

<i>standard input</i>	<i>standard output</i>	<i>picture of the path</i>
2	404240022	



This page is intentionally left blank

Problem L. Triangles

Time limit: 2 seconds
Memory limit: 1024 mebibytes

Consider n integer points on a plane, where n is divisible by 3. The x coordinates are a permutation of integers from 1 to n . The y coordinates are also a permutation of integers from 1 to n .

Construct a configuration of $n/3$ triangles with vertices in the given points: each point must appear as a vertex of one of the triangles. The triangles can be degenerate and can intersect freely.

The score of a configuration is the sum of the sizes of all triangles in it. The size of a triangle is the perimeter of its bounding rectangle. The bounding rectangle is the minimal rectangle with sides parallel to coordinate axes that contains the triangle.

Find a configuration that achieves the maximum possible score.

Input

The first line contains an integer n ($3 \leq n \leq 99\,999$; n is divisible by 3).

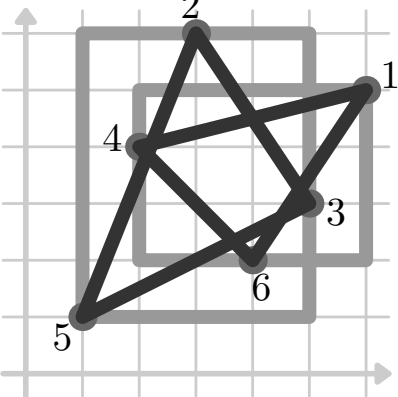
Each of the next n lines contains two integers x and y : the coordinates of one of the points. Each integer from 1 to n appears once among the x coordinates and once among the y coordinates.

Output

On the first line, print the maximum possible score.

Next, print any configuration that achieves this score. For that, print $n/3$ lines with three integers in each: the numbers of points in the input that are the vertices of a triangle. The points are numbered from 1 to n in the input order. Triangles and their vertices can be printed in any order.

Example

<i>standard input</i>	<i>standard output</i>	<i>explanation</i>
6 6 5 3 6 5 3 2 4 1 1 4 2	32 2 3 5 6 1 4	



This page is intentionally left blank



Problem M. Construction Company

Time limit: 12 seconds
Memory limit: 1024 mebibytes

The New Year holidays are over, and your construction company is getting back to work. Unfortunately, some employees are still celebrating...

Today, a sober and b drunk workers came to work for you. Each worker will be available for the whole day. Your company also received m orders for today. Each order is specified by the start and the end of the time span to perform it: $[st, fn]$. Such an order requires a single worker to work on it from moment st to moment fn , inclusive. A worker can complete multiple orders if their time spans do not intersect. Additionally, each order has one of the two types: a simple order can be completed by any worker, while a complex order requires a sober worker.

Will your company be able to handle today's orders?

Input

The first line contains an integer t , the number of test cases ($1 \leq t \leq 2000$). For each test case:

The first line contains three integers: a , b , and m ($0 \leq a, b, m \leq 10\,000$).

Then m lines follow. Each line contains three integers, $type$, st , and fn , describing a simple (if $type = 1$) or complex ($type = 2$) order with the time span $[st, fn]$ ($type \in \{1, 2\}$; $1 \leq st \leq fn \leq 2 \cdot m$). For example, a line `1 2 4` describes a simple order which requires three moments, from second to fourth, both ends included.

The sum of m over all test cases is at most 10 000.

Output

For each test case, print a single line with the answer: "Yes" or "No" (case-insensitive).

Example

<i>standard input</i>	<i>standard output</i>
1 1 1 2 1 1 2 2 2 4	Yes



This page is intentionally left blank